BORDERLESS
<CODING>

Rust for Web3 development

Part 01 · Introduction

Solana · Web3 mental model

Rust & Anchor · Solana Playground

Minimal program in Rust

About me

César Patiño

Senior Web3 & full-stack developer

+10 years of experience
+300 projects

Rust · Solana
Solidity · Hardhat
Python · Node.js · React · TypeScript

Instructor @ Borderless Coding

What you will learn

1. What is the **Solana** blockchain: network, validators, programs, accounts
2. The **Web3** mental model: transactions, state, Web2 analogies
3. The **Rust** and **Anchor** basics: read and edit on-chain code with Anchor
4. How to use the **Solana Playground**: build, deploy, run tests in the browser
5. How to create a **minimal program in Rust**

What is the Solana blockchain

What is the Solana blockchain?

- **Layer-1 network**: validators run the same rules; history is a shared ledger
- **Fast, cheap confirmations** for many apps (good UX for products, games, payments)
- **Programs** (smart contracts) you deploy; users send **transactions** that call **instructions**
- **SOL** pays fees and can be locked as **rent** for storing account data

Solana network diagram

flowchart TB CW[Clients / wallets] -->|" signed transactions "| VC[Validator cluster] VC -->|" blocks + consensus "| BL[Shared ledger] BL --> RT[SVM runtime] RT --> PR[Programs] RT --> AC[(Accounts)] PR -.->|" instructions "| AC

Technologies

Blockchain: Solana

Programming Language: Rust

Framework: Anchor

Client Languages & Tests: JavaScript or TypeScript

Why this ecosystem?

- **Performance & throughput**: room to grow real apps without fighting the chain
- **Strong tooling**: Playground, Anchor, wallets, explorers — short path from idea to deployed program
- **Rust for on-chain logic**: one language for safe, high-performance programs
- **Active developer community**: docs, examples, and standards keep improving

New Solana addresses by year

xychart-beta title "New addresses / year (approx., millions)" x-axis [2020, 2021, 2022, 2023, 2024, 2025] y-axis "Millions" 0 --> 120 bar [2, 14, 24, 58, 105, 98]

Source: The Block — monthly new addresses summed per calendar year (millions, rounded). 2020 from mainnet launch (Mar).

The Web3 mental model

Web3 on Solana: quick map (1/2)

Web3 idea (Solana)	Web2 simile
Transaction	One **request** that either fully succeeds or fully fails
Program	**Stateless service** / shared library (logic only)
Account	**Row** or **document** with an address; holds your app's bytes

Web3 on Solana: quick map (2/2)

Web3 idea (Solana)	Web2 simile
Wallet / signer	**Logged-in user** + permission to act
Cluster (devnet / mainnet)	**Environment** — staging vs production

Key concepts

1. **State** lives in **accounts**
2. **Instructions** change that state when the rules allow it
3. **Rules** are explicit: who signs, which accounts are writable, and how they relate

> *Web2 picture: **API handler + database row** — on Solana the "row" is an account everyone can verify.*

The Rust and Anchor basics

Rust: what you need to know

- **Language**: performance, type safety, basic syntax
- In this module we use it **through Anchor** — you read and edit **small, structured** pieces
- Building blocks: **`struct`**, **`u64`**, **`&mut`**, **`Result`**

> *You do **not** need to be a Rust expert to follow along; patterns repeat in every program*

Anchor framework

Piece	Role
IDL + macros	Typed accounts & instructions
`#[program]`	Your instruction handlers
`#[derive(Accounts)]`	Accounts each instruction needs (Part 2)
vs raw SDK	Less boilerplate · safer defaults

Web2: classic backend

- Implicit **session / env**
- State lives mainly in **one database**

flowchart LR U[Client / browser] -->|HTTP| R[Route / API] R --> S[Service / logic] S --> D[(Central database)]

Web3: blockchain flow

Stateless program; you pass **accounts** every call — no hidden server memory.

flowchart LR W[Wallet / client] -->|" Signed transaction "| RT[Runtime] RT --> P[Program / logic] P --> A1[(Account)] P --> A2[(Account)]

Solana: where does data live?

- **No separate SQL database** behind your program
- **Persistent state** → many **accounts** (each has an address, owner, and data bytes)
- **No global variables** — every instruction receives **accounts + arguments** explicitly

Programs vs accounts

- **Program** = **stateless logic** deployed on-chain (like a shared library the runtime runs)
- **Account** = **durable storage**; clients pass the right accounts into each instruction
- Together: **logic + data**, but the data is **not hidden inside** the program binary

How to use the Solana Playground

Solana Playground

- Browser IDE: playground.solana.com
- **New project → Anchor** — same layout as local Anchor: `lib.rs`, client, tests
- Next slides: **full loop** (build → deploy → run) then **program + client + test** files

Playground — build & deploy

1. **Build** — compiles the on-chain program; fix compiler errors in the panel
2. **Deploy** — uploads the `.so` to Playground's cluster; note the **Program ID**
3. **`declare_id!(...)`** in `lib.rs` must match that **Program ID** (template often prefills it)
4. If you change the id string → **Build** again, then **Deploy** again

Playground — run, client & tests

- **Run test** — runs the test runner (e.g. Mocha) against the **deployed** program
- **`client.ts`** — `pg.program.methods.hello().rpc()` sends a tx; then **confirm** and **`getTransaction`** → **`logMessages`** (see `msg!` output)
- **`anchor.test.ts`** — same instruction from a **test**; assert signature / behavior
- One mental model: **build + deploy once**, then **many runs** (client or tests) hit the same program

How to create a minimal program in Rust

Playground: `lib.rs`

```rust
use anchor_lang::prelude::*;
use solana_program::msg;

declare_id!("Fg6PaFpoGXkYsidMpWTK6W2BeZ7FEfcYkg476zPFLSn");

#[program]
pub mod hello_anchor {
    use super::*;

    pub fn hello(_ctx: Context<Hello>) -> Result<()> {
        msg!("Hello, Solana!");
        Ok(())
    }
}

#[derive(Accounts)]
pub struct Hello {}
```

Link to code on GitHub

Playground: `client.ts`

```
const tx = await pg.program.methods.hello().rpc();

await pg.connection.confirmTransaction(tx, "confirmed");

const details = await pg.connection.getTransaction(tx, {
  commitment: "confirmed",
  maxSupportedTransactionVersion: 0,
});

console.log(details?.meta?.logMessages);
```

Link to code on GitHub

Playground: `anchor.test.ts`

```
describe("hello world", () => {
  it("calls hello", async () => {
    const sig = await pg.program.methods.hello().rpc();
    if (!sig) throw new Error("no signature");
  });
});
```

Link to code on GitHub

After the Playground demo

- **Loop:** edit Rust → build → deploy → invoke → read logs
- **Persistence** still means **accounts** — the program file alone does not store your app state
- **Next:** create and update accounts with Anchor (`init`, `space`, `mut`)

Minimal Rust for Anchor

- **`struct`** — shape of data (on-chain accounts and `Context` later)
- **`u64`** — whole numbers for counters and examples
- **`&mut`** — “mutable reference” = permission to **modify** for the duration of a call
- **`Result<()>`** — success `Ok(())` or an error that **stops** the transaction safely

Example: counter

```
pub struct Counter {
    pub value: u64,
}

fn increment(counter: &mut Counter) {
    counter.value += 1;
}
```

GitHub — replace URL in `slides.html`

- Treat **`Counter`** like a **table row** and **`increment`** like an **UPDATE** inside one transaction

Part 01 recap

- **Solana:** validators, programs, transactions, accounts, SOL / rent
- **Web3 here:**
  - transactions ≈ atomic requests
  - accounts ≈ rows
  - programs ≈ stateless logic
- **Rust + Anchor:** small set of syntax; Anchor adds structure and safety

Next (Part 2)

- `#[program]` handlers in detail
- `Context`
- `#[derive(Accounts)]`
- **`init` / `space`**

> *Programs **read and write accounts**; that is the core skill for building on Solana.*